Membership business — identity resolution and billing sync

A membership business synced its billing platform to HubSpot with the native connector. Every re-signup minted a new record, and ~5,000 members split into duplicates. The fix wasn't a field map — it was an identity model behind middleware.

Paul Maxwell, CEO, RevOps HQ July 2026

ABSTRACT

A multi-location membership business — recurring subscriptions across dozens of sites, plus physical check-ins — wired Stripe into HubSpot with the off-the-shelf connector. It worked until members behaved like members: cancel, come back, and the billing platform issued a brand-new customer ID each time. The native 1:1 sync treated every new ID as a new human, and roughly 5,000 contacts fractured into duplicates with broken subscription history and unusable lifetime value. This is a first-principles walk through the failure most integrations hit but few scope for: identity resolution. It covers why native connectors structurally can't solve it, the bridge-object model that collapses many billing IDs into one person, the custom objects a non-deal business actually needs, and the decision line between an in-portal Custom Code action and real middleware.

1 THE INTEGRATION FAILURE IS IDENTITY, NOT FIELDS

Most integration projects are scoped as field mapping: connect two systems, match their properties, watch the data flow. That framing hides the failure mode that actually breaks CRMs — not *which field goes where*, but *which records are the same person*. Identity resolution is the hard part, and it is almost never in the statement of work.

The tell is simple: ask what happens when the source system issues a new identifier for someone who already exists. If the answer is "a new contact appears," the integration has no identity model, and it is only a matter of volume before the CRM fills with ghosts.

THE FIRST PRINCIPLE

An integration needs an identity model before a field map. Decide what makes two records the same human, where that decision lives, and how a re-used or re-issued ID collapses onto one person. Everything downstream — history, LTV, lifecycle — depends on it.

2 WHY A NATIVE CONNECTOR MULTIPLIES PEOPLE

The billing platform re-issued a customer ID every time a member cancelled and later signed back up. To Stripe those are two customer objects; to the business they are one person with a gap. A native HubSpot–Stripe connector maps one billing customer to one contact — a 1:1 sync with nowhere to say "these two IDs are the same human." So every returning member became a second contact, and about 5,000 records ended up duplicated, their subscription history split across the copies and their lifetime value meaningless.

Two more things a 1:1 connector cannot do surfaced at the same time. **Ordering:** invoice webhooks arrived before the customer record existed, because independent connectors fire with no sequencing. **Non-CRM data:** physical check-ins — a member badging into a location — are usage events with no native connector at all. None of these is a mapping problem; they are all consequences of missing a layer that can hold identity, order, and event data.

3 MODELING IDENTITY: THE BRIDGE OBJECT

The fix is a small idea with large consequences. Between the billing platform and the Contact, add a **bridge object** — one record per billing customer ID — and associate many of them to a single Contact. The Contact becomes the durable human; the bridge records are the aliases the billing system keeps minting. A returning member's new ID becomes another bridge record on the same person, not a new person.

With identity anchored, subscription attributes — membership type, billing interval, start and cancel dates, status — are flattened onto the Contact, so "are they active?" is one property, not a query across duplicates. Lifetime value sums correctly because every payment resolves to one human again.

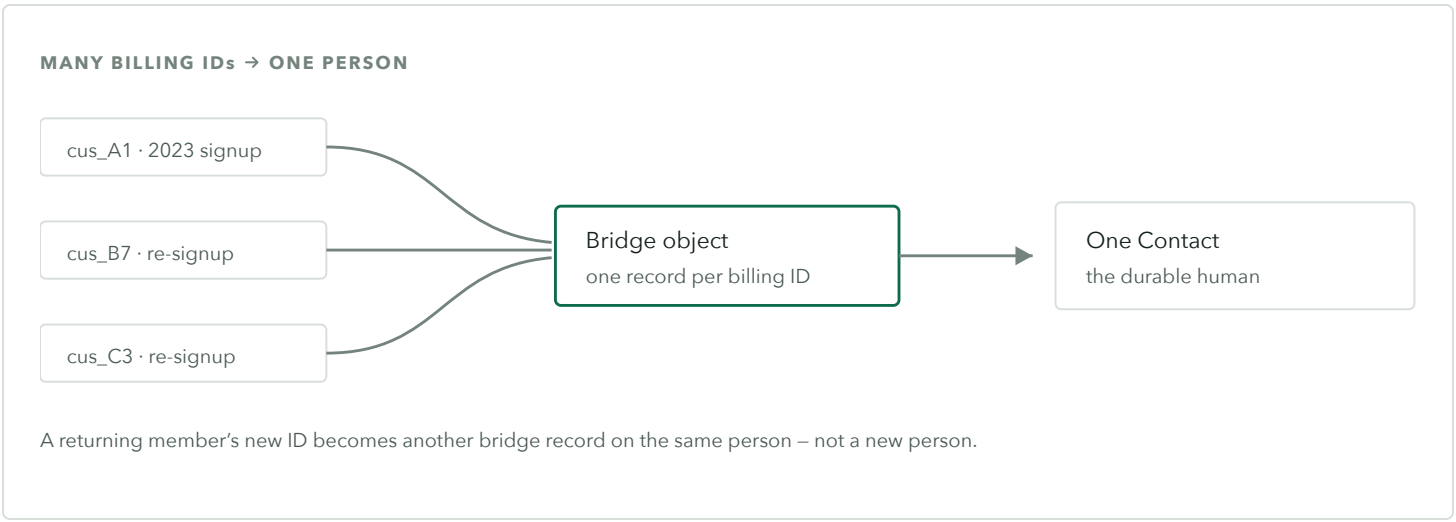

Figure 1. The bridge object holds one record per billing customer ID and associates many of them to a single Contact, so re-issued IDs resolve to one human instead of fracturing into duplicates.

TRANSFERABLE PATTERN

When a source system re-issues IDs, do not map its ID straight onto your primary record. Introduce an alias/bridge object (many source IDs → one entity) and resolve identity there. It is the single most reusable fix in CRM integration.

4 A MEMBERSHIP BUSINESS ISN'T DEAL-SHAPED

HubSpot's defaults assume a deal pipeline. A membership business does not sell deals; it enrolls members and records what they do. Modeled from first principles, the objects are: the **Contact** as the member and system of record for membership status; a **Location** custom object with a location → area → market hierarchy for roll-up reporting; a **Payment** object; and **Visit** records for check-ins. Physical check-ins flow from the access system through the app's database into HubSpot as Visit events — usage signals the CRM otherwise never sees.

None of this is exotic; it is just modeling the business as it is rather than forcing it into contacts-and-deals. The association hierarchy is what makes "members and revenue by location, area, and market" a native report instead of a spreadsheet.

5 THE REAL DECISION: CUSTOM CODE VS. MIDDLEWARE

HubSpot gives you three ways to integrate: a native connector, in-portal **Custom Code workflow actions**, and external **middleware**. The consultative skill is knowing which the problem needs. Native was already shown to fail on identity. Custom Code actions run inside HubSpot and are perfect for light, in-portal logic — but they inherit the portal's execution limits and have no place to hold a durable identity map, sequence out-of-order webhooks, or backfill history at volume.

Here the load — real-time billing webhooks, high-volume check-in events at peak, historical backfill, and dedup logic that must run reliably — pushed past what in-portal code carries. That is the line where middleware earns its place. A single middleware layer (RevOps Connect) became the one path all data flows through: billing, access events, and web signups land there, get ordered, deduplicated against the bridge object, and written to HubSpot — with retries, a field-level audit log, and the ability to re-run a failed sync. One embedded action card on the record even lets an agent cancel, pause, or refund a subscription without leaving HubSpot.

THE DECISION, STATED PLAINLY

Native connector for simple 1:1 sync. Custom Code action for light in-portal logic within HubSpot's limits. Middleware when you need identity resolution, ordering, retries, backfill, or volume beyond the portal. Naming the right one is the difference between a build that holds and one that fills with ghosts.

6 WHAT CHANGED

With identity resolved, the roughly 5,000 fractured members collapsed back to one record each, subscription history reattached, and lifetime value became a number the business could trust again. Membership status lives on the Contact, so lifecycle automation and “win-back” outreach finally target the right people instead of duplicates. Check-in data flowing in as Visit events turned engagement into something reportable — who is actually showing up, by location, area, and market. On the marketing side, seven per-location acquisition workflows collapsed toward a single cloneable template, cutting the maintenance surface.

Figures are representative of the engagement and drawn from RevOps HQ’s delivery experience; the ~5,000-record dedup is the one figure documented in the source.

7 THE MODEL, GENERALIZED

Independent of membership, the exercise repeats on any integration:

- **Model identity before fields.** Decide what makes two records one human, and where that lives.
- **Never sync a re-issuable ID onto your primary record.** Put an alias/bridge object between them.
- **Model the business, not the default.** A non-deal business needs its own objects and an association hierarchy for roll-ups.
- **Pick the integration tier deliberately.** Native / Custom Code / middleware is a decision, not a default — match it to identity, ordering, volume, and backfill.

RELATED STUDIES

The same "model before software" discipline runs through migrating an *RIA off a legacy CRM* (sharding one table into objects), representing a *dealer channel* where the buyer isn't the user, and modeling *project-based revenue* when the deal isn't the work.